05 · Dot Products, Softmax & Attention Math

Source: fanout.sh / inference-eng / 00-03 · ~11 min

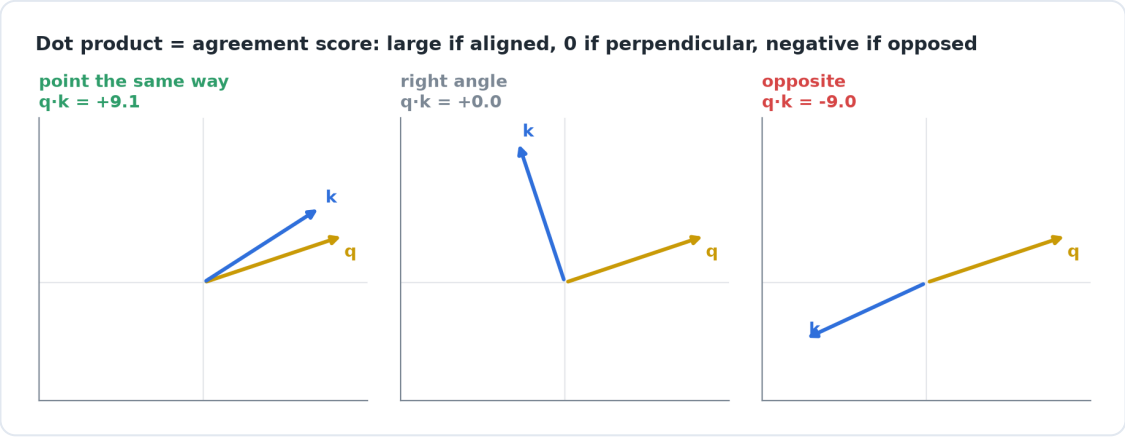
> **IN SHORT**
>
> To write one new token at 4,096 tokens of context, Llama 3 8B's attention does about **4.2 million dot products** (~2.1B operations) but must read about **0.5 GB** of stored keys and values. That's ~4 operations per byte, far below the ~300 an H100 needs, so **decode attention is memory-bound**. The K/V data grows with every token and every user until it outweighs the model. Softmax needs the whole row, which is why fast kernels use a running (online) softmax.

1. The dot product: an agreement score

Multiply two equal-length lists pair by pair, then add:

$$(2,\ -1, 0.5) \cdot (3, 4, 2) = 6 - 4 + 1 = 3$$

- In attention each list has **128 numbers**: 128 multiplies + ~128 adds ≈ **256 operations**.
- The result says how much two vectors **point the same way**:

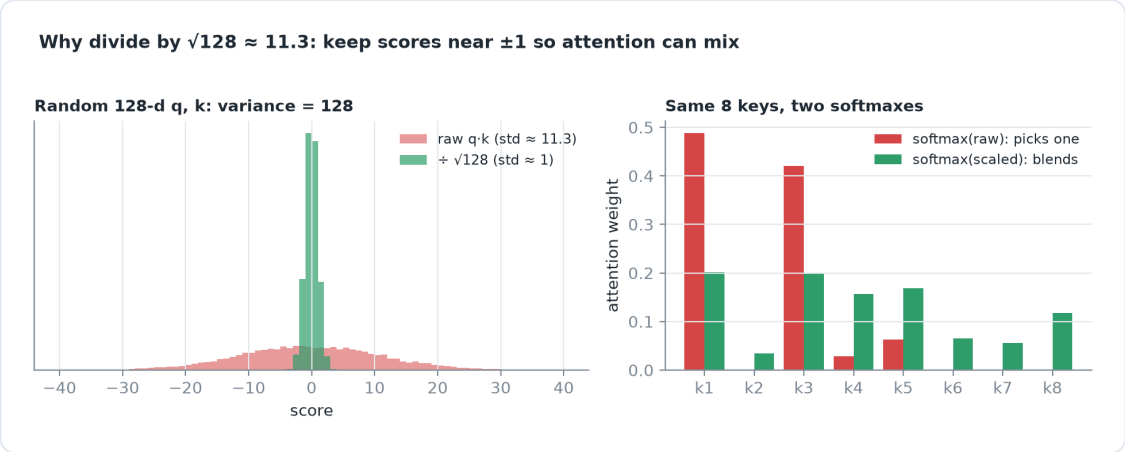


Aligned vectors give a large positive score, perpendicular ones zero, opposed ones negative.

- Attention uses it as a **relevance score**: the current token's **query** asks, each past token's **key** answers, and q·k says how well they match.

Why divide by √128

If query and key entries are random with mean 0 and variance 1, their dot product has **variance = length** (footnote in *Attention Is All You Need*). For 128 numbers, scores spread to about **±11**, and softmax then puts nearly all weight on one token: attention stops blending and just picks.



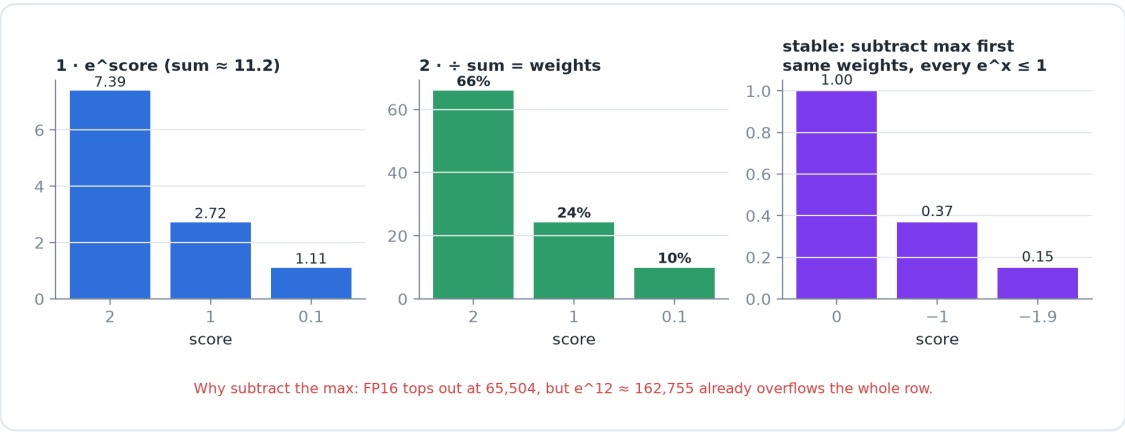
Simulated: raw scores spread ±11 and softmax picks a winner; dividing by √128 ≈ 11.3 brings them to ±1 and the weights blend.

INTUITION

For inference the scaling is nearly free: one multiply per score, and fast kernels fold it into work they already do.

2. Softmax: scores into weights

Softmax makes scores **positive and summing to 1**: raise e to each score, divide by the total.



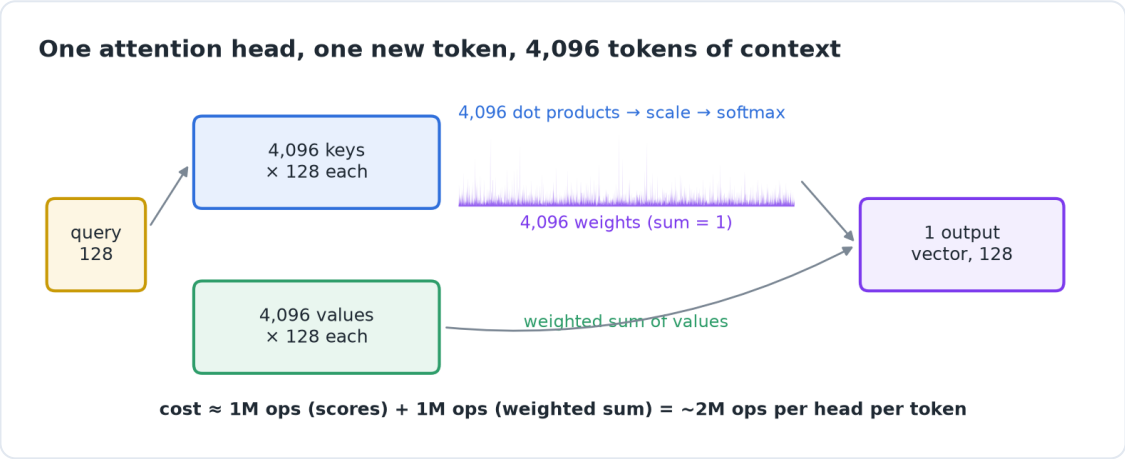
Scores 2, 1, 0.1 → e^x ≈ 7.39, 2.72, 1.11 (sum ≈ 11.2) → 66%, 24%, 10%. The biggest score wins most, but nothing gets zero.

- **Overflow trap:** FP16's largest value is **65,504**, but $e^{12} \approx$ **162,755**. One large score turns the whole row into garbage.
- **Fix:** subtract the row's **maximum** first (2, 1, 0.1 → 0, −1, −1.9). Same weights, and every exponent is ≤ 1.

KEY IDEA

Softmax needs the **max and the sum over the entire row** before any weight is final. This shapes how every fast attention kernel is built (section 6).

3. One head, one new token



Query · 4,096 keys → scale → softmax → weighted sum of 4,096 values → one 128-number summary of the conversation, as seen from the new token.

Work: ~**1M ops** for the scores + ~**1M ops** for the weighted sum = **~2M ops per head**.

4. The bill for one token: ops vs bytes

Scale to the whole model: **32 layers × 32 query heads × 4,096 = 4,194,304 dot products ≈ 2.1B operations**. The weight matrices cost ~**15B** ops per token, so at this context attention's arithmetic is only **~1/8**.

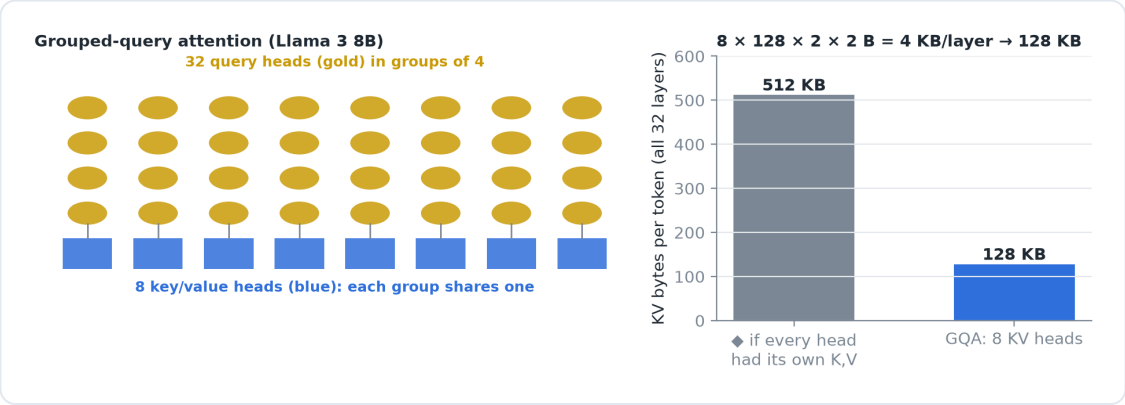
Where 2.1B comes from: two equal halves, the scores and the weighted sum of values (scaling and softmax add only a few million).

$$\text{scores}: 4.19\text{M dots} \times 256 \approx 1.07\text{B} \qquad \text{values}: 32 \times 32 \times 4096 \times 128 \times 2 \approx 1.07\text{B} \qquad \Rightarrow \approx 2.1\text{B}$$

Where ~15B comes from: each weight does one multiply and one add per token, so ~2 ops per parameter. The ~0.5B-parameter input embedding table (128,256 vocab × 4,096) is a lookup, not a matmul, so ~7.5B parameters do work:

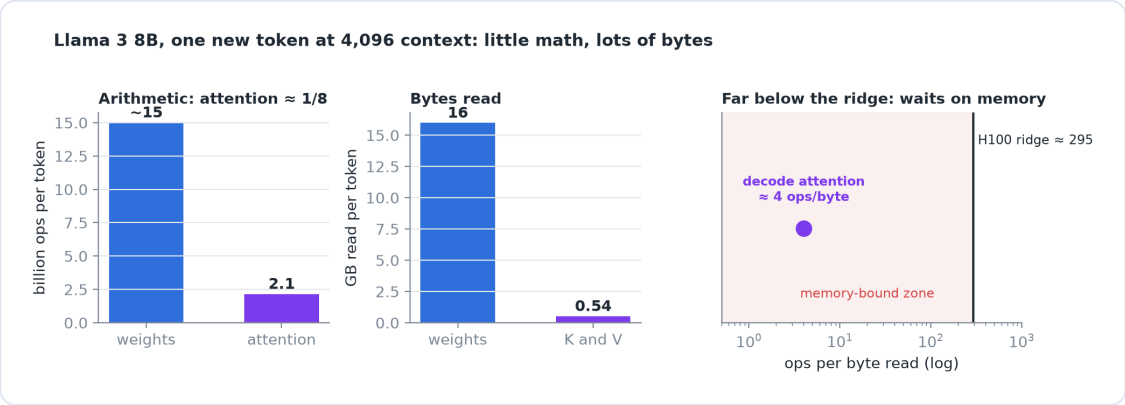
$$2 \times (8.03\text{B} - 0.53\text{B}) \approx 15\text{B} \qquad \frac{2.1}{15 + 2.1} \approx 12\% \approx \frac{1}{8}$$

Now count bytes. Llama 3 uses **grouped-query attention (GQA)**: the 32 query heads share just **8** sets of keys and values.



Four query heads share each K/V head, so each layer stores 8 keys + 8 values per token.

$$8 \text{ heads} \times 128 \times 2\ (\text{K, V}) \times 2\ \text{B} = 4\ \text{KB/token/layer} \quad \times 32 = 128\ \text{KB/token} \quad \times 4096 \approx 0.5\ \text{GB}$$



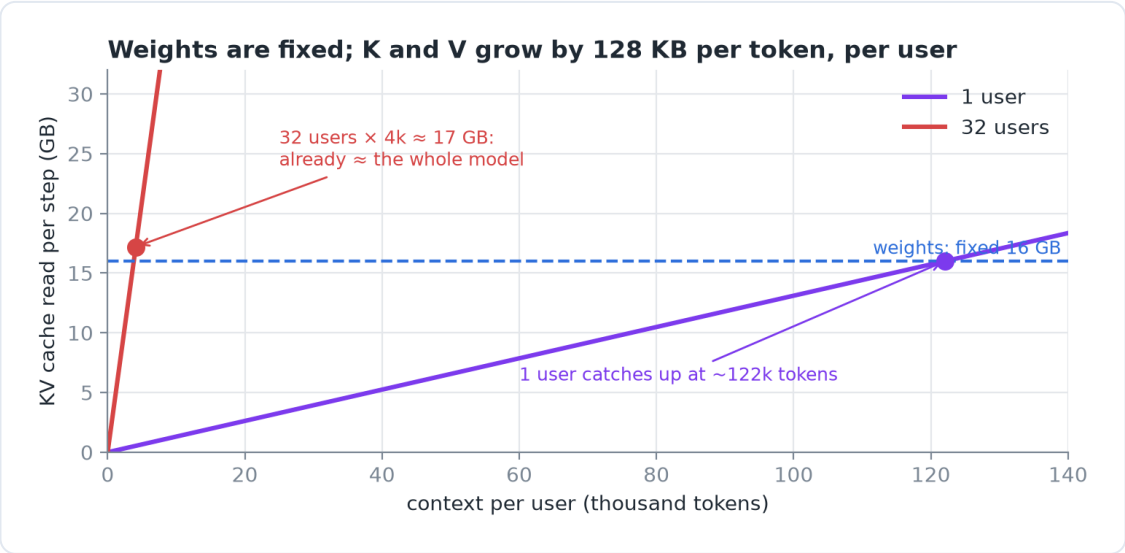
2.1B ops over ~0.54B bytes ≈ 4 ops/byte; an H100 needs ~295 (989 TFLOPS ÷ 3.35 TB/s) to keep its math units busy.

KEY IDEA

Four is nowhere near 300: **decode attention spends almost all its time waiting on memory**.

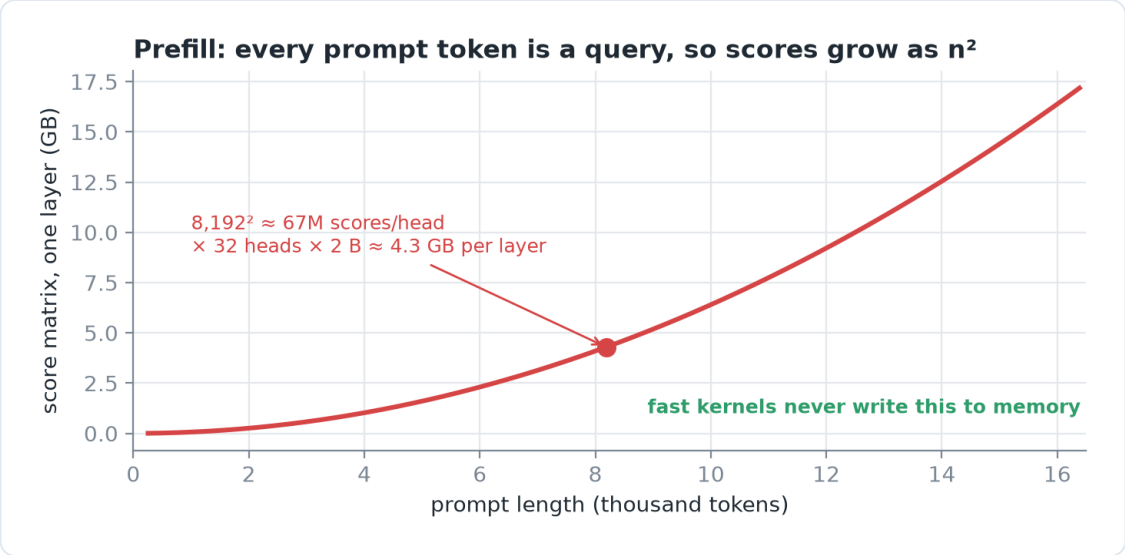
5. The memory bill grows

- **Weights:** a fixed **16 GB**, read once per step however long the conversation.
- **Keys and values:** **+128 KB per token**, per user.



One user's K/V catches the weights at ~120k tokens; 32 users at 4k tokens each already hold ~17 GB.

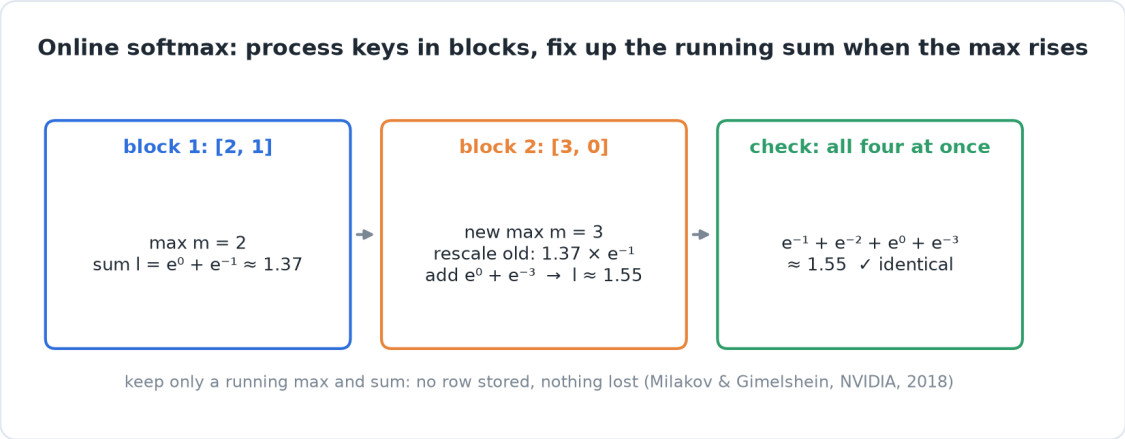
Prefill has a different problem: every prompt token is a query at once, so the scores form an **n × n square**.



An 8,192-token prompt: ~67M scores per head; × 32 heads × 2 bytes ≈ 4.3 GB of scratch per layer.

6. Online softmax: never write the square

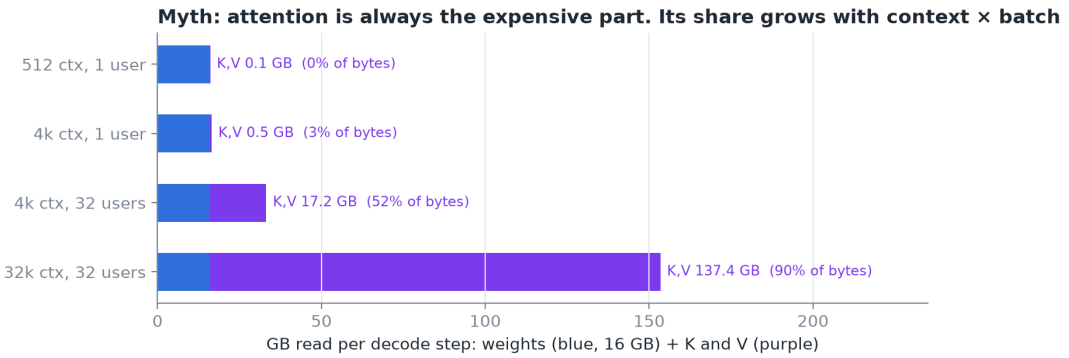
Fast attention kernels walk the keys **in blocks** and keep a **running max and running sum**. When a new block raises the max, rescale the old sum by e ^ (old max − new max), then add the new terms.



Blocks [2, 1] then [3, 0] give a running sum of ~1.55, exactly the full-row result. Nothing stored, nothing lost.

This is the **online softmax** (two NVIDIA engineers, 2018), the foundation of **FlashAttention**; the attention module builds on it.

7. Two misconceptions



At short context and small batch the weights dominate the bytes; attention's share grows with context × batch, mostly through memory.

TRAP

- "Attention is always the expensive part": not at short contexts and small batches.
- "Softmax is cheap, so it doesn't matter": the arithmetic is cheap, but needing the whole row shapes every fast kernel.

8. Recap

Lesson 05 on one card

DOT PRODUCT

agreement score of two vectors
≈ 2 ops per number (256 for d = 128)
divide by √d to keep scores ~±1

SOFTMAX

scores → positive weights summing to 1
subtract the max (FP16 overflows)
needs the whole row, or a running one

DECODE ATTENTION

4.2M dots ≈ 2.1B ops per token
reads 0.5 GB of K,V at 4k context
≈ 4 ops/byte → memory-bound

GROWTH

128 KB per token per user
1 user = weights at ~120k tokens
32 users × 4k ≈ 17 GB